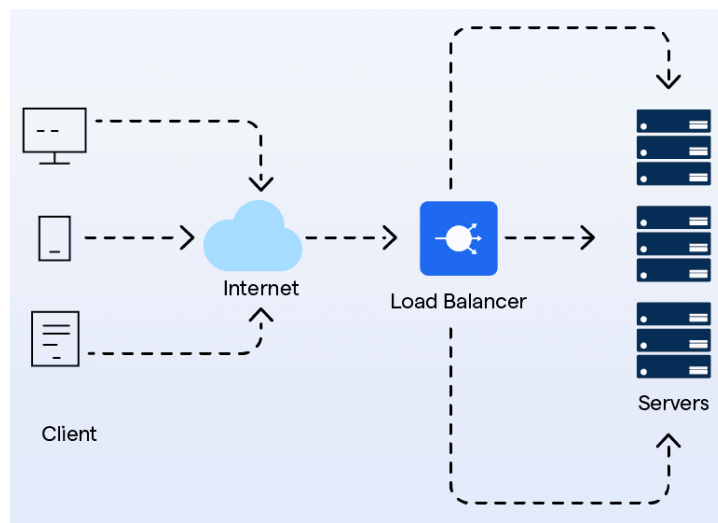


Load Balancing

What is Load Balancing?



Load balancing is the process of **distributing incoming network traffic** (like user requests) **across multiple servers** to ensure:

- No single server is overwhelmed
- Users get faster responses
- Systems stay available and reliable

Why is Load Balancing Important?

Pros and Cons of Load Balancers

Pros	Cons
 Improve uptime	 Geographic limitations in certain load-balancing algorithms
 Scale applications	 Possibility of assigning only a single point of failure
 Boost network and application performance	 Difficulty in keeping up with technological advancements
 Protect applications from network-related attacks	 Delays due to overly complex algorithms
 Make IT operations more flexible	
 Share SSL decryption workloads with the server	

Without load balancing:

- One server could receive **too many requests**, slow down, or even crash
- Others may sit **idle**
- Users may experience **slowdowns** or **downtime**

With load balancing:

- Traffic is **spread evenly**
- All servers work efficiently
- Users get a **better experience**

Visual Example:

Without Load Balancer:

Users → [Single Server]

If that server is overloaded or fails, everything stops.

With Load Balancer:

Users → [Load Balancer] → Server A
→ Server B
→ Server C

Now traffic is **shared**, and failure of one server doesn't stop the whole system.

How Does Load Balancing Work?

The **Load Balancer** sits between the users and your servers.

When a user makes a request, the Load Balancer decides **which server** should handle it.

This decision is made using a **strategy** or **algorithm**.

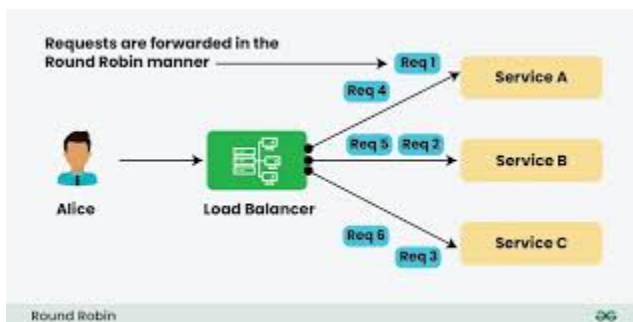
Common Load Balancing Strategies

Here are 3 simple and popular strategies:

1. Round Robin

How it works:

Requests are sent to servers **one after the other**, in a circular order.



Example:

Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A
...and so on.

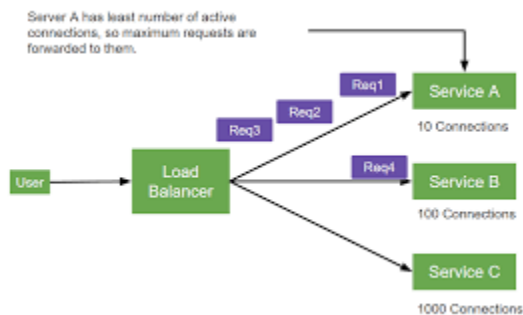
Advantages: Simple, Good for similar server sizes

Disadvantages: Doesn't check server load or response time

2. Least Connections

How it works:

The request goes to the server that has the **fewest active connections** (i.e., the least busy).



Example:

[Server A] → 2 connections
[Server B] → 1 connection
[Server C] → 3 connections
→ New request goes to ****Server B****

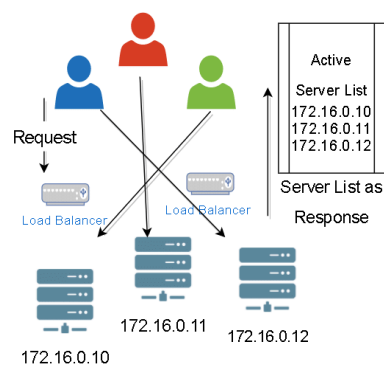
Advantages: Smart — adapts to server load, Good for long-lived connections

Disadvantages: Slightly more complex to implement

3. Random

How it works:

Each request is sent to a **randomly selected server**.



Example:

Request 1 → Server C
Request 2 → Server A
Request 3 → Server B
Request 4 → Server A

Advantages: Simple, Good for evenly matched servers

Disadvantages: Doesn't account for server load, May hit same server repeatedly by chance